

DÍVIDA TÉCNICA EM PROJETOS DE SOFTWARE: revisão bibliográfica e um novo modelo quantitativo

Anonymous Author(s)

Abstract

Este trabalho empreende uma revisão sistemática do conceito de dívida técnica em engenharia de software, a qual se manifesta em problemas estruturais no código que, quando negligenciados, **prejudicam** a manutenibilidade e a evolução das aplicações. Adicionalmente, propõe-se um modelo quantitativo que integra métricas estáticas de qualidade do código, fornecidas pelo *SonarQube*, a um fator temporal inspirado na curva de esquecimento de *Ebbinghaus*. Para a validação da métrica, realizou-se a avaliação de múltiplos repositórios de código aberto em várias linguagens interpretadas, mediante a coleta de dados históricos de *commits* e a análise da relação entre o tempo decorrido e o esforço estimado de manutenção. O modelo proposto foi calibrado por regressão de mínimos quadrados e confrontado com abordagens convencionais de mensuração de dívida técnica. Os resultados demonstram que a incorporação do componente temporal intensifica a correlação entre a métrica apresentada e o esforço real de correção, otimizando a priorização de atividades de refatoração.

CCS Concepts

• **Software and its engineering** → **Software evolution.**

Keywords

dívida técnica; análise de código; gestão de projetos de software; segurança do código; manutenção de software; sonarqube; desenvolvimento de software; modelo quantitativo; repositórios de código aberto; métricas de qualidade de código

1 Introdução

Dentro da área de engenharia de *software*, é rotineiro encontrar sistemas de tecnologia que estão em fases de desenvolvimento ou sustentação. Tais sistemas apresentam desafios tecnológicos inerentes como depreciação da tecnologia utilizada, aumento da complexidade do código fonte, assim como desafios de gestão de projetos, como a priorização do desenvolvimento de novas funcionalidades. Muitas vezes, tal desenvolvimento é priorizado em detrimento da melhor organização e manutenção do código fonte. No entanto, quanto mais o código é alterado sem corrigir os problemas inerentes, mais a qualidade do *software* tende a reduzir [12].

De forma geral, a dívida apresenta um ponto de atenção para os projetos de software e sua quantificação é fundamental. Atualmente, existem poucos *frameworks* ou ferramentas que conseguem mensurar e identificar os principais pontos de tal dívida de forma direta, acionável e que permita ao time uma priorização eficaz das correções. Este trabalho foca em analisar os resultados de vários projetos ao longo do tempo utilizando a ferramenta referência para este tipo de análise - *SonarQube* [1] - e avaliar um modelo para descrever tal montante/esforço. O objetivo é auxiliar na tomada de decisão se uma funcionalidade nova pode ser priorizada ou se é mais adequado realizar uma melhoria sistêmica do código fonte. E,

claro, um ponto importante é que o modelo tenha uma componente temporal, tentando replicar a ideia de uma dívida convencional, normalmente acompanhada de juros.

Com um modelo de mensuração de envelhecimento e esforço da dívida estabelecido, é possível identificar com maior clareza a dívida técnica instalada no código fonte e o aumento do esforço para resolvê-la ao longo do tempo, além de facilitar a priorização e resolução dos problemas. Além disso, pode auxiliar várias equipes de desenvolvimento a justificar a priorização de refatoração do código em detrimento de novas codificações, assim como levantar as principais questões na dificuldade de implementação de certas funcionalidades devido ao estado atual do código fonte. Tal modelo é de alta relevância, pois a falta de tal ferramenta/métrica pode levar a uma priorização inadequada de tarefas, resultando em *software* de baixa qualidade e com alta dificuldade em sua manutenção.

1.1 Metodologia

O foco do trabalho é encontrar um modelo que correlacione o tempo passado desde a identificação do problema e a quantidade de esforço aplicado para resolvê-lo. Para isso, serão escolhidos vários repositórios de código aberto (para servir como amostra com boa significância estatística), ferramentas de análise e metodologias de validação e construção de modelos matemáticos. Tal modelo deve auxiliar na tomada de decisão dos gestores do projeto do que deve ser priorizado naquele determinado instante.

Existem várias ferramentas de controle de versão de código fonte, mas o *GIT* é de longe a mais utilizada pela comunidade. Por exemplo, uma das maiores plataformas de hospedagem de código, o *GitHub* (que trabalha com o *GIT*) recebeu em 2022 mais de 3,5 bilhões de *commits* e foram criados mais de 85 milhões de novos repositórios públicos e privados [8]. Com isso, ele foi escolhido como ferramenta de controle de versão principal desta análise, já que através dele é possível contabilizar as alterações feitas linha a linha de cada um dos arquivos versionados e, o *GitHub*, como fonte para escolha dos repositórios, além de ter se tornado uma das plataformas mais citadas em artigos [5].

Foram avaliadas várias formas de selecionar do *GitHub* quais repositórios fariam parte das análises e passariam pelo processo de identificação de problemas. Como o objetivo do trabalho consiste em identificar métricas que refletissem uma grande massa de dados, foram selecionados repositórios seguindo os seguintes critérios (com suas devidas explicações):

- **Linguagem:** foram priorizados repositórios em linguagem interpretada, como *JavaScript*, *TypeScript*, *Python* e *PHP*. Isso permitiu uma análise mais rápida das informações pois, para tais linguagens, o *SonarQube* não precisa compilar o projeto para identificar os problemas (o que poderia causar uma complicação muito maior caso em alguns pontos do código o projeto não compilasse ou, no caso de projetos mais antigos, incompatibilidade ou indisponibilidade de



SDKs e bibliotecas). Além disso, tais linguagens estão na lista das mais populares segundo o TIOBE Index;

- **Quantidade de Commits:** com o objetivo de ter mais dados históricos ao decorrer da evolução do código, foram escolhidos projetos com pelo menos 500 *commits* do *git* ou mais de 5 anos de existência;
- **Quantidade de Estrelas:** dentro da plataforma do GitHub, existe uma funcionalidade que permite um usuário favoritar um projeto. Normalmente essa métrica reflete a popularidade deste código e, para ter uma métrica mais fiel e buscar repositórios que têm mais chance de sofrerem alterações, foram escolhidos projetos que possuem pelo menos 500 estrelas;
- **Quantidade de Contribuidores:** muitos dos projetos são mantidos por uma ou duas pessoas, o que pode tornar o processo de gestão da dívida mais frágil. Assim, foram priorizados repositórios com pelo menos 5 contribuidores;
- **Quantidade de Problemas:** durante a execução da análise pela ferramenta SonarQube, foi identificado que sua *API* de recuperação de *issues* limita os resultados em 10.000 (independente do formato de paginação ou tamanho/edição da instância). Com isso, todos os projetos analisados apresentam menos de 10.000 problemas ativos em qualquer momento no tempo (pois, caso apresentassem mais, não seria possível avaliar corretamente a criação e resolução de todos eles).

Para a análise de tempo de correção, foram trazidos alguns conceitos do trabalho de *Bakardzhiev*[2] que se baseiam na ideia de que a capacidade de aplicar o conhecimento depende da memória humana e da acessibilidade ao conhecimento necessário. Assim como no conceito da curva de esquecimento de *Ebbinghaus*[4], quanto mais tempo passa desde que o conhecimento foi adquirido ou utilizado, maior a dificuldade em recuperá-lo de maneira eficiente, o que pode aumentar a taxa de retrabalho dentro de um código fonte. Além disso, foram levantadas métricas de quando o problema foi identificado e quanto tempo um determinado trecho apresentou dívida técnica. As métricas serão geradas para *commits* intercalados entre períodos de 14 dias (simulando um tempo de *sprint* padrão). Este período foi escolhido não só para reduzir a quantidade de análises necessárias (pois existem repositórios com mais de 50 mil *commits* e, considerando um tempo de análise de 4 minutos para cada um, levaria 139 dias ininterruptos para obtermos o resultado) mas também por representar um intervalo comum no contexto de desenvolvimento de *software*. E claro, utilizando o *GIT*, é possível visualizar as modificações consolidadas em um arquivo entre esses dois pontos no tempo.

Para cada ponto no tempo, a análise registrou as seguintes informações para os problemas encontrados:

- A data e o *commit* em que o problema foi identificado;
- A data e o *commit* em que o problema foi corrigido (quando for corrigido);
- A quantidade de linhas problemáticas do código;
- Informações sobre o problema como: severidade, categoria, linha de início e linha final do trecho;
- As métricas de qualidade de código atualizadas, incluindo a complexidade ciclomática, *code smells* e *bugs*.

Após a geração de todos os dados relacionados aos problemas do código, se iniciou a etapa de análise e construção de visualizações das informações coletadas. A análise quantitativa da relação entre o tempo decorrido e o esforço necessário para resolver a dívida técnica será realizada por meio do cálculo das correlações de *Pearson* [11] e *Spearman* [13].

A busca por um modelo que expresse a correlação entre o esforço investido no desenvolvimento de um código e o período em que ele foi escrito e corrigido, exige uma análise dos fatores que podem justificar a influência do tempo nesse contexto. Para isso, são levantadas as seguintes justificativas:

- **Impacto do Tempo na Compreensão do Código:** à medida que o tempo passa, o conhecimento sobre determinados trechos de código tende a se deteriorar, especialmente se não houver revisitas regulares. Isso aumenta a dificuldade de realizar alterações seguras e eficientes [12];
- **Relação Direta com Dívida Técnica:** a dívida técnica inclui elementos que se acumulam devido à dificuldade crescente de manutenção. Uma métrica baseada no tempo reflete de forma mais realista as barreiras enfrentadas ao trabalhar com código antigo (o que o modelo proposto neste trabalho pretende identificar)[6];
- **Alteração das Regras de Negócio:** a dinâmica dos sistemas de software exige que as regras explicitadas no código fonte sejam constantemente revisadas e atualizadas. Trechos de código que permanecem inalterados por longos períodos tornam-se mais suscetíveis a mudanças substanciais, uma vez que as regras do sistema evoluem para atender a novas demandas e flexibilidades [3].

E, para buscar representar este parâmetro de tempo no modelo, foi escolhida a curva de esquecimento proposta por *Ebbinghaus* [4], adaptada ao contexto de software. A escolha foi baseada nos seguintes fatores:

- **Base Teórica Sólida:** A curva de *Ebbinghaus* descreve matematicamente o declínio exponencial da memória ao longo do tempo, sendo amplamente reconhecida e replicada em diversos contextos, conforme *Murre*[10].
- **Aplicação em Engenharia de Software:** Estudos como o de *Fekete*[7] demonstraram que a fórmula pode ser ajustada para quantificar a dificuldade em alterar trechos de código antigos, com sucesso na prática.
- **Personalização para Contexto de Software:** A fórmula permite ajuste de coeficientes para refletir as particularidades dos projetos de software, aumentando sua aplicabilidade e precisão no cálculo de dificuldade de manutenção [7].

A publicação de *Ebbinghaus* também incluiu uma equação para aproximar sua curva de esquecimento, descrita em (1):

$$R = \frac{100k}{(\log(t))^c + k} \quad (1)$$

Aqui, *R* representa a quantidade de conhecimento retido expressa como uma porcentagem, e *t* representa o tempo em minutos, contando a partir de um minuto antes do fim da aprendizagem. As constantes *c* e *k* são 1,25 e 1,84, respectivamente. A quantidade de conhecimento retido é definida como a porcentagem de informação

ainda lembrada em relação à aprendizagem inicial. Uma retenção de 100% indicaria que todos os itens ainda eram conhecidos desde a primeira tentativa. Uma retenção de 75% significaria que 75% do conhecimento original ainda é lembrado.

Com isso, é possível construir um modelo inicial que captura tais parâmetros temporais e busca estimar um provável esforço para resolução. A proposta deste modelo é descrita abaixo (2) com todos seus componentes, e é escopo do trabalho encontrar os melhores pesos e valores de coeficientes para equilibrar os resultados.

Dificuldade na Resolução (Dr)

$$Dr = \alpha \times We \quad (2)$$

Onde:

- α = Coeficiente de ponderação para problemas do código;
- We = Equação de Ebbinghaus adaptada (3) (complemento da (1));

Peso We :

$$We = 100 - \frac{100k}{(\log(t))^c + k} \quad (3)$$

Para ajustar os parâmetros do modelo proposto, tanto α quanto c e k , foi utilizado o método de mínimos quadrados aplicado aos dados coletados. O principal objetivo dessa abordagem é minimizar a diferença entre os valores observados da variável que apresenta a maior correlação com o tempo para a correção e os valores previstos pelo modelo ajustado. A escolha do método de mínimos quadrados é justificada por sua capacidade de encontrar os melhores parâmetros para ajustar a curva ao comportamento dos dados, mesmo em cenários onde os dados apresentam variabilidade significativa, como explorado por *Levie*[9]. Os valores iniciais de c e k foram escolhidos a partir fórmula original de Ebbinghaus.

Para garantir a qualidade do ajuste, diferentes métricas de avaliação foram calculadas durante o processo, o Erro Quadrático Médio (MSE), o Coeficiente de Determinação (R^2) e a Análise de Resíduos, buscando mais indicadores de que o modelo encontrado representa bem os dados encontrados.

1.2 Resultados Preliminares

Foram avaliados 100 repositórios, distribuídos entre as principais linguagens não compiladas do mercado, como demonstrado na tabela 1. As análises foram realizadas entre os meses de Dezembro/2024 e Janeiro/2025 utilizando a versão 10.7 do SonarQube e a versão 6.2.1 do SonarScanner.

Linguagem	Quantidade de Projetos
JavaScript	42
TypeScript	35
PHP	12
Python	11

Table 1: Quantidade de repositórios por linguagem

A análise dos projetos coletados permitiu a consolidação de informações relevantes sobre a atividade, o volume de alterações no código e a popularidade de cada um, conforme detalhado na Tabela 2. Observa-se, por exemplo, a presença de projetos com mais de 400

contribuidores ativos e mais de 200 mil estrelas, valores significativamente superiores às médias observadas de 41 mil estrelas e 266 contribuidores, respectivamente, evidenciando alta popularidade. No entanto, ao avaliar o número de problemas ativos (problemas estes que permaneceram sem resolução na última análise pela ferramenta), é possível constatar que nenhum é superior a 10 mil. Este limite, contudo, decorre de uma restrição do *SonarQube*, que impede a recuperação de listas de problemas superiores a essa quantidade. Consequentemente, alguns projetos (além dos 102 inicialmente considerados) que apresentaram um número de problemas acima desse limite foram excluídos da análise. A inclusão desses projetos resultaria em listas de problemas incompletas, comprometendo a coerência da análise estatística do histórico dos mesmos.

Métrica	Média	Mediana	Min.	Max.
Nº de Commits	5.629	3.885	105	35.650
Nº de Contribuintes	254	263	5	466
Nº de Estrelas	44.931	41.615	686	208.262
Nº de Prob. Resolvidos	3.546	1.287	3	35.731
Nº de Prob. Ativos	1.670	904	4	9.820
Qnt. de Linhas	63.087	38.875	78	313.502
Data de Criação	02/2016	03/2016	08/2006	01/2024

Table 2: Métricas dos repositórios coletados

Após a coleta inicial dos dados, iniciou-se a etapa de análise exploratória, cujo objetivo era identificar padrões que relacionam o tempo de resolução de problemas com alguma das variáveis analisadas. Para isso, dentro dos dados iniciais, foram aplicados os seguintes filtros:

- **Dias para resolver menor que 2000:** para remover ruídos de problemas que ficaram muito tempo sem resolução e não impactar a análise geral;
- **Arquivos excluídos:** durante a coleta de métricas, muitos arquivos foram marcados como excluídos. Isso pode acontecer caso ele tenha sido removido do código fonte ou renomeado incorretamente. Logo, para uma análise mais precisa, foram removidos tais arquivos;

Ao avaliar o resultado das correlações de *Pearson* e *Spearman*, dentre várias informações coletadas, destaca-se uma métrica calculada através da quantidade de linhas alteradas no arquivo analisado dividida pela quantidade de linhas totais do arquivo (resultado este que já é naturalmente normalizado) e ela recebeu o nome neste trabalho de Alteração Relativa ao Contexto (ARC). Mesmo apresentando correlações baixas com Dias para resolver (variável que indica o número de dias transcorridos entre a identificação e a resolução do problema), sendo 0,24 na *Pearson* e 0,28 no *Spearman*, é a métrica com os maiores valores encontrados, o que justifica sua escolha para a continuação das análises. O coeficiente de *Spearman* sugere uma correlação moderada de 0,28, indicando uma relação monotônica entre o tempo de resolução e a ARC, coerente com a ideia de que o aumento no percentual de alterações demanda mais tempo para resolução. Já o coeficiente de *Pearson* aponta uma correlação positiva mais fraca, com valor de 0,24, o que pode indicar que a relação não é estritamente linear.

Com o intuito de identificar possíveis relações dos valores da variável ARC, foi construído um gráfico que apresenta estatísticas do tempo necessário para resolver problemas agrupados por intervalos de valores de ARC, destacando tanto as médias quanto as medianas dos tempos registrados. Para que o resultado seja mais interessante visualmente, os dados foram divididos em intervalos de alteração de 10% (ou 0.1), calculando a média e mediana para cada um deles. O resultado dessa análise é apresentado na Figura 1.

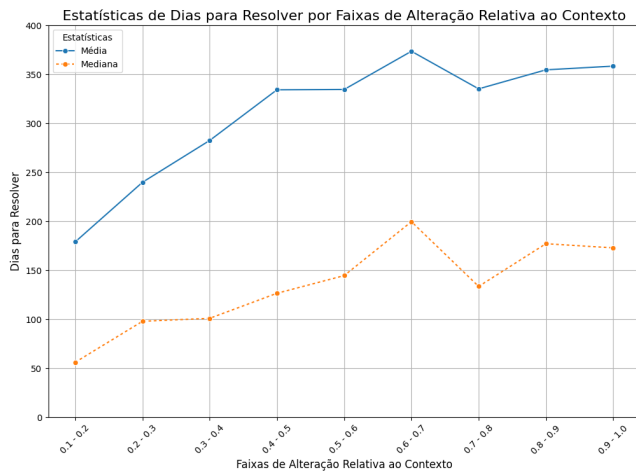


Figure 1: Média e Mediana de Dias para Resolver e ARC

A Figura 1 ilustra um crescimento progressivo no tempo médio de resolução (representado pela linha contínua azul) à medida que os intervalos de ARC aumentam. Esse comportamento sugere uma relação entre o percentual de alterações realizadas no código e o esforço temporal necessário para resolver os problemas, corroborando a hipótese inicial de que maior Alteração Relativa ao Contexto está associada a maiores tempos de correção.

Por outro lado, as medianas (linha tracejada laranja) apresentam um comportamento mais irregular, com oscilações marcadas por picos e quedas em determinados intervalos. Esse padrão menos uniforme sugere que, embora a média capture a tendência geral de crescimento no tempo de resolução, a distribuição dos tempos é bastante dispersa, principalmente em valores mais elevados de ARC. Esse comportamento pode ser indicativo da variabilidade intrínseca no esforço de correção, refletindo fatores contextuais, como a complexidade dos problemas abordados e a familiaridade dos desenvolvedores com o trecho de código modificado.

Para investigar essa variabilidade com maior profundidade, foi gerado um gráfico de dispersão (figura 2), no qual o eixo X representa o tempo necessário para resolver os problemas (em dias) e o eixo Y, os valores de Alteração Relativa ao Contexto.

A análise do gráfico (figura 2) revela que os dados não apresentam um padrão claro ou uma correlação evidente entre as variáveis. Os pontos estão amplamente espalhados, indicando alta variabilidade. Esse comportamento sugere que a relação entre ARC e o tempo de resolução pode estar sofrendo influência de uma série de fatores externos, como o nível de ruído nos dados, variáveis não analisadas, ou até mesmo diferenças no fluxo de manutenção e nas prioridades dos projetos e contribuidores.

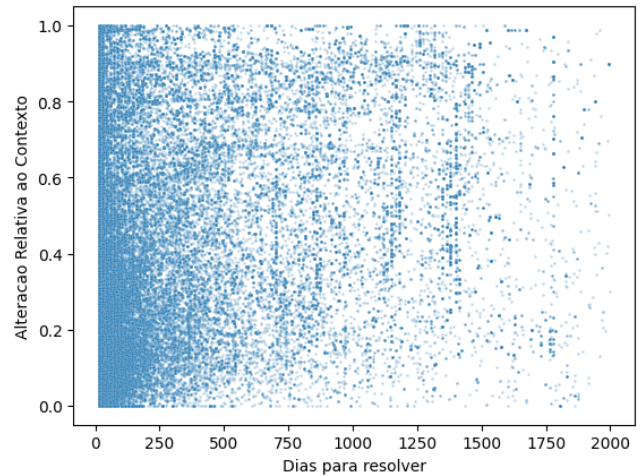


Figure 2: Gráfico de Dispersão de Dias por ARC

Essa ausência de uma correlação clara no gráfico de dispersão reforça a complexidade da análise de dívida técnica, destacando a importância de considerar múltiplos fatores e contextos específicos de cada projeto. Apesar disso, as tendências gerais observadas nas médias e medianas indicam que a Alteração Relativa ao Contexto desempenha um papel relevante, embora não exclusivo, na determinação do esforço necessário para resolver problemas.

No entanto, para seguir com a análise, foram calculadas as correlações de *Pearson* e *Spearman* em subconjuntos de dados, separando os problemas por projetos. Essa abordagem permitiu avaliar se determinados repositórios apresentavam padrões mais consistentes entre as variáveis elencadas. Ao aplicar um filtro, foram selecionados apenas os repositórios com uma correlação de *Pearson* superior a 0,4, representando um nível moderado de associação. Como resultado, identificou-se que apenas 20% dos projetos analisados (cobrindo aproximadamente 40% dos dados totais, totalizando 44.977 dos 116.299 pontos iniciais) atenderam a esse critério, evidenciando uma relação mais robusta entre as variáveis nesses casos/projetos/repositórios específicos.

A escolha de filtrar os dados baseou-se em três justificativas principais:

- **Relevância Estatística:** Correlações abaixo de 0,4 indicam associações fracas, frequentemente influenciadas por ruídos ou variações aleatórias. Ao focar nos casos mais correlacionados, foi possível priorizar cenários com maior potencial de revelar padrões significativos;
- **Redução do Impacto de Outliers:** Projetos com baixa correlação podem estar influenciados por dados atípicos, como alterações pontuais ou problemas isolados. A filtragem garantiu maior consistência nos resultados e evitou vieses decorrentes de comportamentos extremos;
- **Foco em Casos Representativos:** Projetos com correlações mais fortes refletem melhor a dinâmica real de acúmulo e resolução de dívida técnica, oferecendo um campo de análise mais adequado para validar o modelo e fornecer *insights* práticos.

Embora a aplicação do filtro tenha reduzido a amostra geral, os casos selecionados apresentaram maior qualidade analítica, permitindo explorar com mais profundidade a relação entre o tempo de resolução e a Alteração Relativa ao Contexto. A Figura 3 ilustra os dados de dispersão dos projetos selecionados, cujos valores de correlação são para *Pearson* 0.55 e, para *Spearman* 0.63. Essas visualizações reforçam que, nos projetos com alta correlação, a relação entre as variáveis é mais clara e consistente, confirmando a adequação dessa abordagem para análise detalhada.

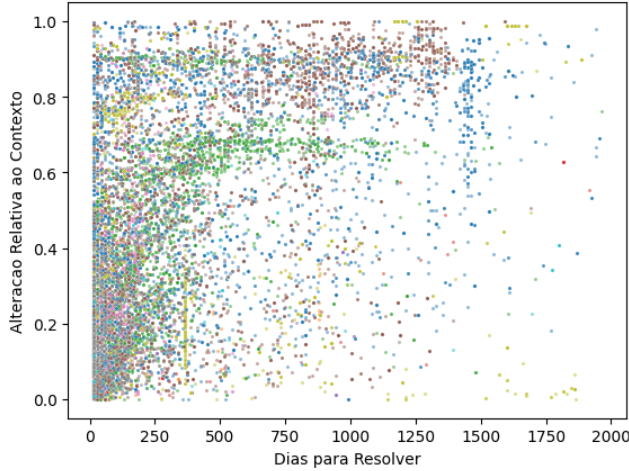


Figure 3: Dispersão de pontos de projetos com correlação > 0.4

Por último, com os dados filtrados, foi construído um *boxplot* representado na Figura 4. A Figura indica que há um aumento gradual no tempo mediano necessário para a resolução à medida que o ARC se aproxima de valores mais elevados. Isso sugere que alterações que impactam uma maior proporção do código relativo ao seu contexto estão associadas a maior esforço de correção. No entanto, a elevada dispersão observada nos *boxplots*, especialmente em intervalos superiores a 0,7, destaca uma variabilidade significativa nos tempos de resolução, possivelmente influenciada por fatores externos, como a complexidade do problema ou o contexto específico do projeto. Essa análise reforça a hipótese inicial de que o aumento do impacto relativo das alterações dificulta o processo de manutenção, ao mesmo tempo que evidencia a necessidade de considerar múltiplos fatores no modelo proposto.

Com a filtragem aplicada e a seleção dos projetos mais correlacionados, foi possível identificar padrões visuais mais claros entre o tempo de resolução e a Alteração Relativa ao Contexto (ARC). A análise dos gráficos de dispersão e das matrizes de correlação para esses projetos demonstrou relações mais consistentes, permitindo uma base mais sólida para modelagem matemática.

2 Ajuste do Modelo

A partir dos dados filtrados, foi realizada a modelagem da curva utilizando o método dos mínimos quadrados (*least squares*), com o objetivo de ajustar os parâmetros de forma a minimizar os erros entre os valores observados e os previstos pelo modelo. Foram

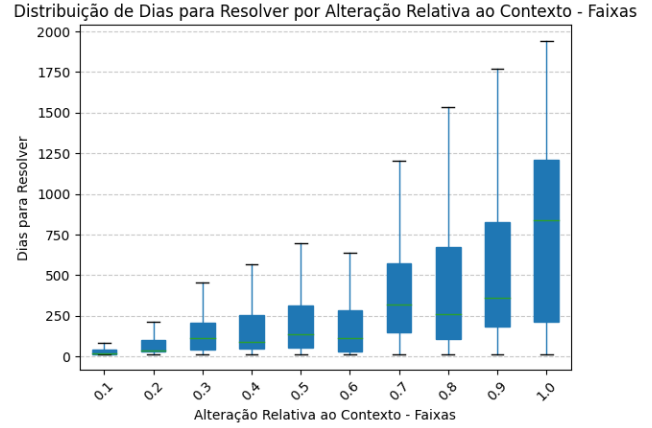


Figure 4: Gráfico de *boxplot* das faixas de ARC por Dias para Resolver

adotados parâmetros iniciais com valores heurísticos: $\alpha = 0.5$, $c = 1.25$, e $k = 1.84$. Esses valores foram escolhidos com base em estudos anteriores sobre a curva de esquecimento de *Ebbinghaus*, adaptada ao contexto da dívida técnica.

O processo de ajuste resultou nos seguintes valores otimizados para os parâmetros do modelo, cuja curva pode ser visualizada na Figura 5:

- α : 0.2782
- c : 1.5034
- k : 650.8592

Resultando na equação:

$$Dr = 0.278 \times \left(100 - \frac{100 \times 650.9}{\log(t)^{1.503} + 650.9} \right) \quad (4)$$

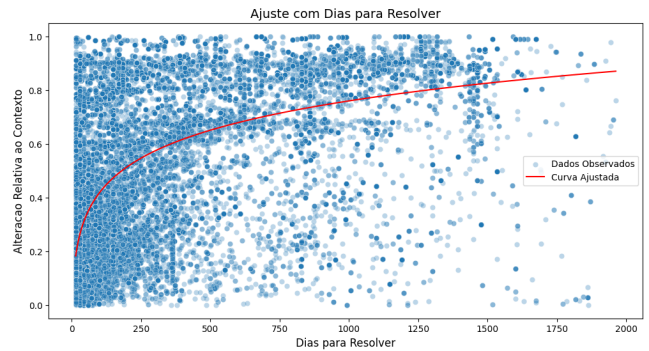


Figure 5: Curva de aumento do esforço ajustada para os dados

Esses novos parâmetros refletem a adaptação do modelo ao comportamento observado nos dados, ajustando a curva para capturar as nuances da relação entre ARC e o tempo de resolução. Sendo assim, o resultado **Dr** reflete a porcentagem de reescrita de um arquivo de código fonte.

Para avaliar a qualidade do ajuste, foram calculadas métricas de desempenho baseadas na diferença entre os valores previstos e os valores observados:

- **Erro Quadrático Médio (MSE):** 0.0647 - O MSE mede a média dos erros ao quadrado entre os valores observados e os previstos. O valor baixo obtido indica que o modelo foi eficaz em minimizar os desvios durante o treinamento.
- **Coefficiente de Determinação (R^2):** 0.3861 - O R^2 avalia a proporção da variabilidade nos dados que é explicada pelo modelo ajustado. Embora um valor de 0.3861 indique uma explicação parcial, é importante considerar que, dada a natureza dos dados e o impacto de ruídos e fatores externos, esse resultado é significativo para um modelo exploratório.

2.1 Análise de Resíduos

A análise de resíduos, cujo resultado é demonstrado na Figura 6 foi conduzida para verificar a qualidade do ajuste e identificar possíveis padrões não capturados pelo modelo:

- **Média dos Resíduos:** -0.0003 - Uma média próxima de zero indica que o modelo não apresenta vieses sistemáticos, com os erros distribuídos de forma equilibrada entre previsões superestimadas e subestimadas.
- **Desvio Padrão dos Resíduos:** 0.2543 - O desvio padrão fornece uma medida da dispersão dos resíduos. O valor obtido reflete uma variabilidade moderada, sugerindo que o modelo, embora funcional, ainda pode ser refinado para capturar aspectos mais sutis da relação entre as variáveis.

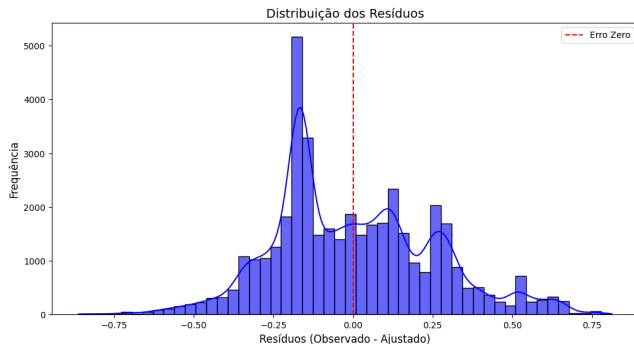


Figure 6: Resíduo da curva treinada

3 Interpretação e Implicações

Os resultados obtidos demonstram que o modelo é capaz de capturar padrões significativos na relação entre o tempo de resolução e a Alteração Relativa ao Contexto, especialmente nos casos filtrados. No entanto, os valores de R^2 e do desvio padrão dos resíduos indicam que há outros fatores relevantes que não foram considerados no modelo atual.

Esse ajuste inicial estabelece uma base sólida para futuras iterações do modelo, permitindo a incorporação de variáveis adicionais que possam melhorar sua capacidade preditiva. Além disso, os parâmetros ajustados, especialmente o valor de k , indicam que o impacto do tempo no esforço de resolução é mais acentuado do que

os valores heurísticos iniciais sugeriam, destacando a relevância do tempo como fator crítico na gestão da dívida técnica.

4 Conclusão

Os resultados indicaram que a dificuldade de resolver problemas aumenta conforme o tempo passa, representada essa característica pela fórmula adaptada da curva de esquecimento de *Ebbinghaus*, que modela o impacto temporal sobre a dificuldade de manutenção. O modelo ajustado, dado pela equação:

$$Pr = 0.278 \times \left(100 - \frac{100 \times 650.9}{\log(t)^{1.503} + 650.9} \right), \quad (5)$$

captura a relação entre o tempo decorrido desde a introdução do problema e o esforço necessário para resolvê-lo, destacando que a dificuldade aumenta de forma não linear. As métricas de desempenho, como o Erro Quadrático Médio (MSE) de 0,0647 e o coeficiente de determinação (R^2) de 0,3861, indicam que o modelo fornece uma base inicial válida para explorar o comportamento da dívida técnica, mesmo considerando os ruídos e a variabilidade nos dados.

Os resultados também revelaram que apenas 20% dos projetos analisados apresentaram correlações mais robustas entre o tempo de resolução e a ARC, representando aproximadamente 40% dos pontos de dados totais. Esse filtro foi fundamental para identificar padrões consistentes e reforçar a aplicabilidade do modelo em cenários específicos. Contudo, os gráficos de dispersão mostraram que a variabilidade dos dados permanece significativa, sugerindo que outros fatores, como a complexidade dos problemas ou características específicas dos projetos, também influenciam o comportamento da dívida técnica.

Estudos futuros podem ampliar o modelo em diversas direções. Em primeiro lugar, a inclusão de variáveis adicionais, como a frequência de *commits*, o número de contribuidores e o custo financeiro associado à manutenção, poderia aumentar a capacidade preditiva do modelo. Em segundo lugar, a aplicação em projetos de diferentes linguagens de programação, especialmente aquelas que requerem compilação, poderia validar a generalização do modelo proposto. Por fim, abordagens de aprendizado de máquina poderiam ser exploradas para identificar automaticamente os fatores mais relevantes e melhorar as previsões em cenários complexos.

Em resumo, o trabalho contribuiu para o entendimento da **dinâmica** da dívida técnica, propondo um modelo matemático fundamentado que destaca a relação entre o tempo e o esforço de manutenção. A fórmula encontrada fornece uma base sólida para explorar essa problemática em diferentes contextos, enquanto os resultados reforçam a necessidade de abordagens sistemáticas e proativas para lidar com a dívida técnica, prevenindo que “juros” elevados comprometam a sustentabilidade e a qualidade dos projetos de software.

References

- [1] Paris C. Avgeriou, Davide Taibi, Apostolos Ampatzoglou, Francesca Arcelli Fontana, Terese Besker, Alexander Chatzigeorgiou, Valentina Lenarduzzi, Antonio Martini, Athanasia Moschou, Ilaria Pigazzini, Nyyti Saarimaki, Darius Daniel Sas, Saulo Soares De Toledo, and Angeliki Agathi Tsintzira. 2021. An Overview and Comparison of Technical Debt Measurement Tools. *IEEE Software* 38, 3 (May 2021), 61–71. doi:10.1109/MS.2020.3024958
- [2] D.V. Bakardzhiev. 2024. What is Knowledge Work. <https://docs.kedehub.io/kede/what-is-knowledge-work.html>

- [3] Keith H. Bennett and Václav T. Rajlich. 2000. Software maintenance and evolution: a roadmap. In *Proceedings of the Conference on The Future of Software Engineering*. ACM, Limerick Ireland, 73–87. doi:10.1145/336512.336534
- [4] H. Ebbinghaus. 1913. *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University. <https://books.google.com.br/books?id=oRSMDF6y3l8C>
- [5] Emily Escamilla, Martin Klein, Talya Cooper, Vicky Rampin, Michele C. Weigle, and Michael L. Nelson. 2022. The Rise of GitHub in Scholarly Publications. doi:10.48550/arXiv.2208.04895 arXiv:2208.04895 [cs].
- [6] Davide Falessi and Andreas Reichel. 2015. Towards an open-source tool for measuring and visualizing the interest of technical debt. In *2015 IEEE 7th International Workshop on Managing Technical Debt (MTD)*. IEEE, Bremen, Germany, 1–8. doi:10.1109/MTD.2015.7332618
- [7] Anett Fekete, Eötvös Loránd University, Faculty of Informatics, Egyetem tér 1-3., 1053 Budapest, Hungary. Email address: afekete@inf.elte.hu, Zoltán Porkoláb, and Eötvös Loránd University, Faculty of Informatics, Egyetem tér 1-3., 1053 Budapest, Hungary. Email address: gsd@inf.elte.hu. 2023. Field Experiment of the Memory Retention of Programmers Regarding Source Code. *Studia Universitatis Babeş-Bolyai Informatica* 68, 1 (July 2023), 71–82. doi:10.24193/subbi.2023.1.05
- [8] Carroll Jordan. 2023. The global developer community. <https://octoverse.github.com/2022/developer-community>
- [9] Robert De Levie. 2000. Curve Fitting with Least Squares. *Critical Reviews in Analytical Chemistry* 30, 1 (Jan. 2000), 59–74. doi:10.1080/10408340091164180
- [10] Jaap M. J. Murre and Joeri Dros. 2015. Replication and Analysis of Ebbinghaus' Forgetting Curve. *PLOS ONE* 10, 7 (July 2015), e0120644. doi:10.1371/journal.pone.0120644
- [11] Karl Pearson. 1901. I. Mathematical contributions to the theory of evolution. –VII. On the correlation of characters not quantitatively measurable. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character* 195, 262-273 (Jan. 1901), 1–47. doi:10.1098/rsta.1900.0022
- [12] Ken Power. 2013. Understanding the impact of technical debt on the capacity and velocity of teams and organizations: Viewing team and organization capacity as a portfolio of real options. In *2013 4th International Workshop on Managing Technical Debt (MTD)*. IEEE, San Francisco, CA, USA, 28–31. doi:10.1109/MTD.2013.6608675
- [13] Spearman. 1094. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology* 15, 1 (1094), 72–101.